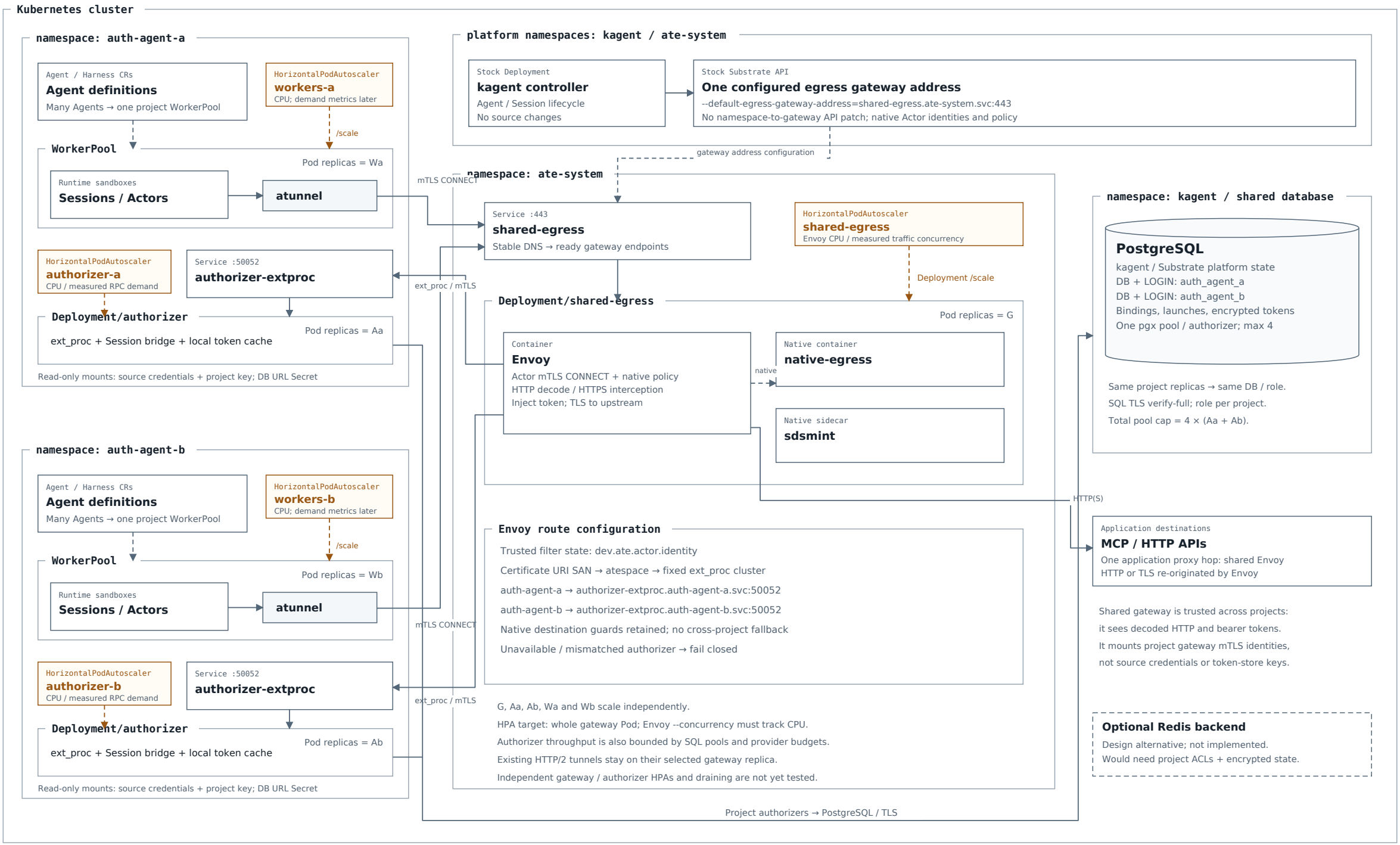
Kubernetes cluster
namespace: auth-agent-a
Agent / Harness CRs
Agent definitions
Many Agents → one project WorkerPool
HorizontalPodAutoscaler
workers-a
CPU; demand metrics later
/scale
WorkerPool
Pod replicas = Wa
Runtime sandboxes
Sessions / Actors
atunnel
HorizontalPodAutoscaler
authorizer-a
CPU / measured RPC demand
Service :50052
authorizer-extproc
Deployment/authorizer
Pod replicas = Aa
ext_proc + Session bridge + local token cache
Read-only mounts: source credentials + project key; DB URL Secret
namespace: auth-agent-b
Agent / Harness CRs
Agent definitions
Many Agents → one project WorkerPool
HorizontalPodAutoscaler
workers-b
CPU; demand metrics later
/scale
WorkerPool
Pod replicas = Wb
Runtime sandboxes
Sessions / Actors
atunnel
HorizontalPodAutoscaler
authorizer-b
CPU / measured RPC demand
Service :50052
authorizer-extproc
Deployment/authorizer
Pod replicas = Ab
ext_proc + Session bridge + local token cache
Read-only mounts: source credentials + project key; DB URL Secret
platform namespaces: kagent / ate-system
Stock Deployment
kagent controller
Agent / Session lifecycle
No source changes
Stock Substrate API
One configured egress gateway address
--default-egress-gateway-address=shared-egress.ate-system.svc:443
No namespace-to-gateway API patch; native Actor identities and policy
gateway address configuration
namespace: ate-system
mTLS CONNECT
Service :443
shared-egress
Stable DNS → ready gateway endpoints
HorizontalPodAutoscaler
shared-egress
Envoy CPU / measured traffic concurrency
Deployment /scale
ext_proc / mTLS
Deployment/shared-egress
Pod replicas = G
Container
Envoy
Actor mTLS CONNECT + native policy
HTTP decode / HTTPS interception
Inject token; TLS to upstream
native
Native container
native-egress
Native sidecar
sdsmint
HTTP(S)
Envoy route configuration
Trusted filter state: dev.ate.actor.identity
Certificate URI SAN → atespace → fixed ext_proc cluster
auth-agent-a → authorizer-extproc.auth-agent-a.svc:50052
auth-agent-b → authorizer-extproc.auth-agent-b.svc:50052
Native destination guards retained; no cross-project fallback
Unavailable / mismatched authorizer → fail closed
mTLS CONNECT
ext_proc / mTLS
G, Aa, Ab, Wa and Wb scale independently.
HPA target: whole gateway Pod; Envoy --concurrency must track CPU.
Authorizer throughput is also bounded by SQL pools and provider budgets.
Existing HTTP/2 tunnels stay on their selected gateway replica.
Independent gateway / authorizer HPAs and draining are not yet tested.
Project authorizers → PostgreSQL / TLS
namespace: kagent / shared database
PostgreSQL
kagent / Substrate platform state
DB + LOGIN: auth_agent_a
DB + LOGIN: auth_agent_b
Bindings, launches, encrypted tokens
One pgx pool / authorizer; max 4
Same project replicas → same DB / role.
SQL TLS verify-full; role per project.
Total pool cap = 4 × (Aa + Ab).
Application destinations
MCP / HTTP APIs
One application proxy hop: shared Envoy
HTTP or TLS re-originated by Envoy
Shared gateway is trusted across projects:
it sees decoded HTTP and bearer tokens.
It mounts project gateway mTLS identities,
not source credentials or token-store keys.
Optional Redis backend
Design alternative; not implemented.
Would need project ACLs + encrypted state.